Desafío 2

Amigos por siempre, Ude@book

Semestre:2026-2

Por: Jonathan Sanmartín Garro

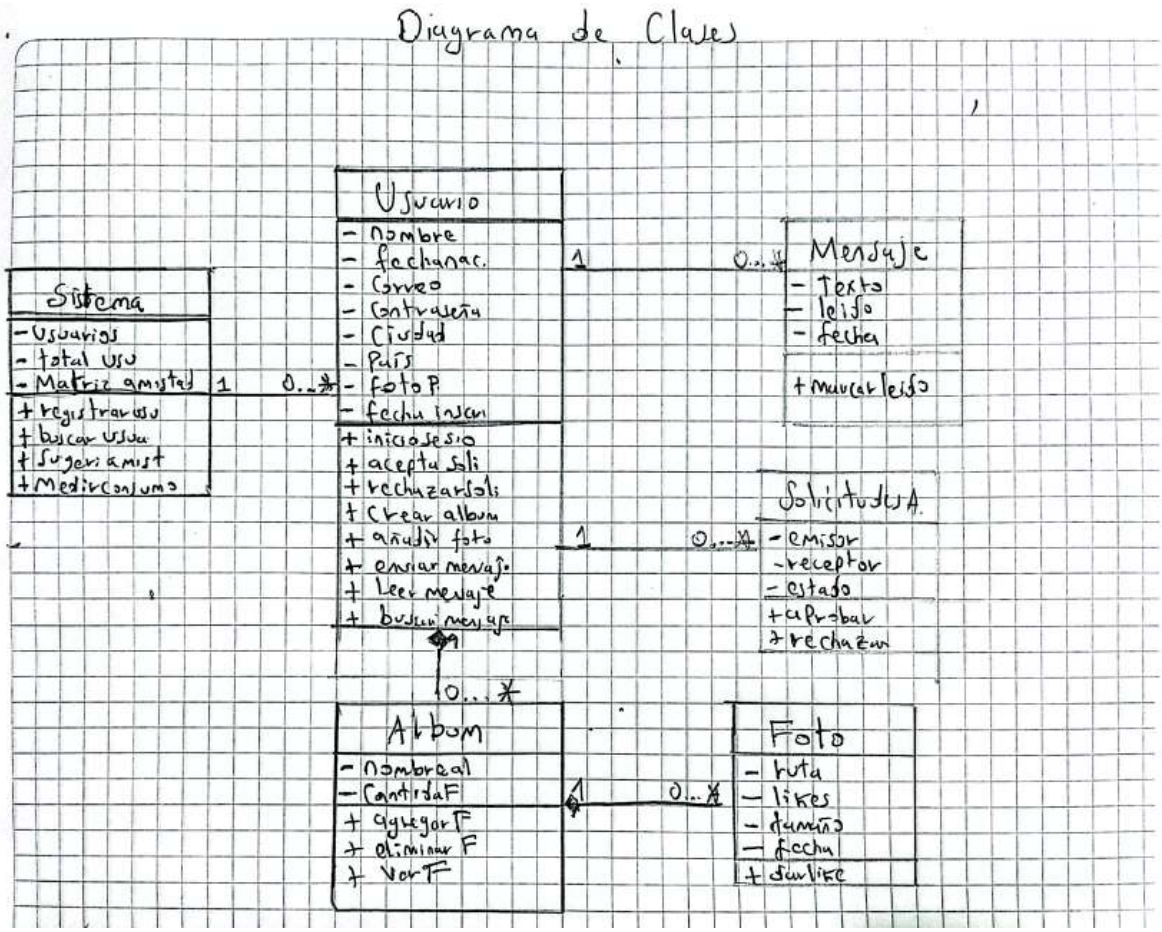
1. Análisis

El desafío proponía realizar una aplicación tipo Facebook mediante la programación orientada a objetos en C++, requería usuarios, amistades, solicitudes, mensajes, álbumes y fotos, también almacenar información, consultar sugerencias y medir los recursos, como sugerencia del profesor en clase podíamos usar matriz de adyacencia para relaciones de amistad y los usuarios con arreglos dinámicos de punteros.

La mejor manera de abordar un desafío de esta magnitud era dividirlo en clases para que fuera más organizado a la hora de desarrollarlas y después integrar, el uso de la memoria dinámica fue fundamental para mantener coherencia en el desarrollo y administrar las reservas de memoria mediante la ejecución, en la implementación priorice la prueba con 1.000 usuarios, las sugerencias de amistad y el consumo de memoria, no se usaron contendores ni ningún tipo de librerías adicionales a las presentadas en clase.

2. Diagrama de clases

Las 6 clases del UML inicial se mantuvieron hasta el final, sistema, usuario, solicitud amistad, mensajes, álbum y fotos, se anexaron solo 2 adicionales, Nodo Foto para las fotos que la idea era enlazarlas de esa manera, y la lista de fotos que tendría una lista enlazada de fotos.



3. Organización de archivos

Todo se hizo usando headers y cpp, en los .h se declararon clases y métodos y en los CPP las implementaciones, al final quedaron solo se dejo en el main el menú y el llamado la demostración.

4. Estructuras seleccionadas

Se implemento arreglo dinámico de punteros a usuario representado por sus respectivos Usuario**. Se usaron las variables cantidadUsuarios y capacidadUsuarios, para cuando lleguen a su limite se amplie a uno mayor copiando la información y su respectiva liberación.

Para las amistades se uso una matriz boleana para validar las relaciones, siendo false 0 no son amigos, true 1 son amigos, se hizo de manera simétrica porqué es una relación es bidireccional, para validar que se cumplen se actualizaban en ambas direcciones, esto permite consultar si son amigos y ayuda a calcular los amigos en común.

También se crearon las clases Nodo y Lista para fotos para almacenar mediante enlaces con nodos, cada nodo tiene una referencia a una foto y como segundo elemento un puntero al siguiente nodo, al final no termine la implementación para la demostración final.

5. Algoritmos esenciales

Para el registro de solicitudes de amistad el programa busca quién emitió la solicitud y quién la recibió que estén registrados, luego valida que existen, no son la misma persona, ya no sean amigos, no tengan solicitudes pendientes, si se cumplen las condiciones se reserva el espacio sino existe y se genera la solicitud y queda en pendiente.

Para aceptar las solicitudes se buscar quien la emite y quien recibe, se revisa la solicitud que no sean amigos y se actualizando la matriz en ambas direcciones y el objeto pasa aceptada.

En las sugerencias primero revisamos que usuario necesitamos, su índice dentro de la matriz y se recorre el arreglo de registrador y se descarta él mismo y sus amigos, para cada uno que no sea amiga se calcula cuantos tienen en común con la matriz de adyacencia, los que tienen coincidencias se almacenan temporalmente y se orden de mayor a menor y se muestran las sugerencias entre 1 y 20.

Para elegir el mejor candidato se recorren todos los posibles y se guarda el índice del que tiene la mayor cantidad de amigos en común si hay varios con la misma cantidad toma el primero que encontró, luego se aplica el método de envío de solicitudes para registrar la nueva.

6. Recursos y eficiencia

Se incorporaron contadores para la cantidad de iteraciones en lagunas funcionalidades porqué no se terminó, se uso long long por referencia en las conectadas para acumular las directas e indirectas y se construyó método para contabilizar la memoria base y estructural pero no toda la real.

En la prueba con 1000 usuarios, se evidenció 12 sugerencias, mejor candidato se seleccionó el usuario4, solicitud enviada sí, iteraciones de sugerencias 997.090, iteraciones del mejor candidato 997.000, iteraciones de envío 6, memoria de la estructura 1’018,464, base usuarios 256.000 y base contabilizada para un total de 1’274.464 bytes.

Para realizar esto implica recorrer los usuarios del candidato considerar, por eso crece muy rápido, el resultado corresponde a las estructuras y objetos incluidos, no es la medición completa de reservas internas, variables temporales y parámetros.

7. Problemas en el desarrollo

Lo primero fue la extensión, aunque se hicieron los prototipos y métodos no se pudo desarrollar completamente, la matriz de amistad se actualizara de manera simétrica, las validaciones para solicitudes duplicadas, muchos contadores y como sumarlos, reconocer la importancia del uso de la memoria dinámica para el desarrollo del proceso.

8. ¿Cómo se desarrolló?

Se hizo análisis con varias lecturas, se elaboró el diagrama de clases, se implementaron las clases básicas, se construyó la liza enlazada para fotos, se implemente el arreglo dinámico de usuarios, se creó la matriz de adyacencia, se desarrollaron las sugerencias de amistad por amigos en común, se implemento el envío y aceptación de solicitudes, se incorporaron contadores y calculo parcial de memoria, se construyó el escenario con 1.000 usuarios y se realizó el menú con la demostración.

9. Alcance y limitaciones

Se realizó el almacenamiento dinámico de usuarios en memoria, representé amistades mediante matriz, consulta y establecer amistades, enviar solicitudes, ver y aceptar, validar que no esten duplicadas o invalidas, generar y ordenar las solicitudes de amistad, seleccionar el mejor candidatos, demostración, medición parcial y menú para lo implementado.

Pendiente, carga y actualizar archivos permanentes, autenticar usuarios, visualizar aleatoria de fotos, integración de álbum, privacidad, carrusel y likes, envío y recepción de mensajes, búsqueda de mensajes por fecha, medición integral de todo, además del menú completo.

10. Conclusiones

Con POO se aprendió a organizar un programa de manera estructural con clases, métodos y objetos separando sus responsabilidades.

Se aprendió a usar matrices de adyacencia para administrar usuarios, estableces amistades y generar sugerencias todo acompañado de punteros y memoria dinámica que ya veníamos trabajando en clase.

Aunque no se terminó, se pudo realizar la prueba de los 1.000 usuarios que cuando lo leí me pareció casi inalcanzable pero se logró, se fortaleció la capacidad de análisis, el orden y el funcionamiento por estructuras.